

UNIVERSIDADE FEDERAL DE SÃO CARLOS
UFSCar Departamento de Computação

Trabalho – Banco de Dados

Tema: Sistema de Streaming de Conteúdo Digital

Nathalia Cristina Santos – 795698

Gabriel Luiz Antonio da Silva - 833230

Yara Vieira de Lima – 824341

Descrição Geral

O trabalho consiste no desenvolvimento de um projeto completo de banco de dados para um **sistema de streaming**, semelhante a plataformas de distribuição de conteúdo digital (filmes, séries, documentários, livros, etc.).

Parte 1 – Modelagem e Implementação Inicial

• Descrição do problema e requisitos de dados

Um sistema de streaming de livros digitais, que o usuário deve fazer uma conta para ter acesso aos conteúdos do sistema. O sistema contará um catálogo de livros que será categorizado pelo nome do livro, autor(es), data de publicação, editora, coleção, gênero e popularidade(baseado nas avaliações dos usuários).

Um usuário possui um ID_usuario(código), nome, sexo, telefone, data de nascimento e idade, email.

Se o usuário for um usuário premium (paga assinatura), ele terá acesso ilimitado aos livros do sistema, fora o acesso offline disponível e sem anúncios durante o uso. Se for um usuário básico (não paga assinatura), ele terá acesso limitado aos recursos, como limite de 5 livros ao mês, fora ser limitado ao acesso somente online do sistema, e com anúncios periódicos durante o uso.

Após ou durante a leitura de algum livro, o usuário (premium ou básico) poderá escrever avaliações sobre o livro. As avaliações poderão ser baseadas em quantidade de estrelas (0 a 5 estrelas), ou, em texto, limitado a 200 caracteres. As avaliações serão públicas. Cada livro mostrará a média das notas que eles são avaliados, sendo uma das métricas de busca do sistema.

Um administrador controlará o sistema, com a função de adicionar e deletar livros da biblioteca. Um administrador possui um ID_adm(código), nome, sexo, telefone, data de nascimento e idade, email.

Será disponibilizado que o usuário veja o histórico de leitura, permitindo a busca de até os últimos cem livros lidos pelo usuário, disponibilizando as datas e nomes das últimas leituras. Ficará disponível também uma estatística de quantos livros completos o usuário leu, quantos ele está lendo no momento e qual a porcentagem de leitura do livro atual.

O usuário premium pode visualizar o histórico de pagamento, que contém valor pago pela assinatura, data dos pagamentos anteriores, e do próximo. Armazenará os dados bancários de cada usuário.

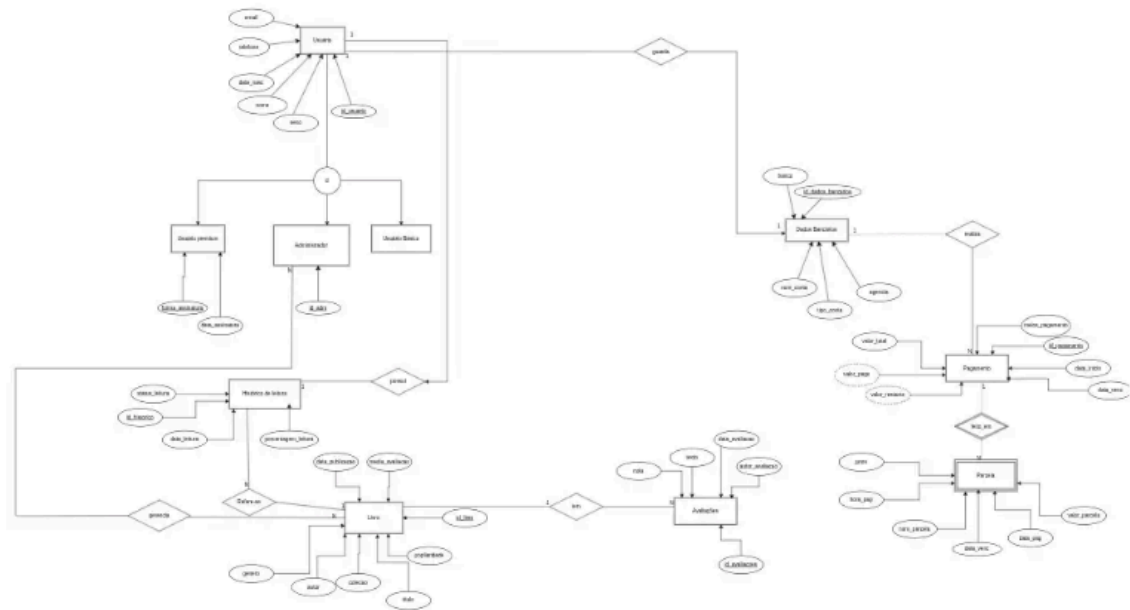
O pagamento será de forma mensal ou anual, função definida na assinatura, e poderá ser feito por meio de débito, crédito e pix. Ele pode ser dividido em parcelas, com pix e débito são somente uma parcela, e com crédito podendo ser divididos em até 12 parcelas. Cada parcela terá um histórico de pagamentos, que contém datas e horários de pagamento, juros e o valor pago em cada pagamento/parcela.

• Projeto Conceitual

Visualização na íntegra em:

https://drive.google.com/file/d/1jjMzTbPjehLn7_7QQobz43CiOxDCE1NW/view?usp=sharing

Foi utilizado a plataforma draw io para fazer o desenvolvimento do projeto conceitual do sistema de streaming.



• Projeto Lógico

fk = foreign key,

wk= weak key,

pw = primary key

usuario = { **pk:id_usuario**, email,telefone,data_nasc, nome,sexo};

usuario_premium ={**pk/fk : id_usuario**,forma_assinatura,data_assinatura}

administrador = {**pk/fk : id_usuario**,id_admin}

usuario_basico ={**pk/fk: id_usuario**}

historico_leitura = {**pk: id_historico**, **fk: usuario**, **fk:livros**, status_leitura, data leitura}

livro = {**pk:id_livro**, titulo, popularidade, colecao, autor, genero, data_publicacao, media_avaliacao};

avaliações = {pk: id_avaliacoes, fk:livros_ava, nota,texto, data_avaliacao, autor_avalacao}

dados_bancarios = {pk: id_dados_bancarios, fk:user, banco, num_conta, tipo_conta, agencia}

pagamento={pk: id_pagamento, fk:dados_banca, meios_pagamento, data_inicio, data_venc, valor_total}

parcela = {pk:{fk: id_pagamento, wk: num_parcela}, juros, hora_pag, data_venc, data_pag, valor_parcela}

gerencia = { pk : {fk:id_usuario_admin, fk:livros}}

Discussões:

- Necessidade de um id_admin: será que é necessário usar uma chave para identificar o administrador?
- Atributos derivados não são armazenados em colunas, eles devem aparecer no modelo relacional?
- Os relacionamentos 1:1 e 1:N foram feitos usando o método da chave estrangeira.
- Pequenas mudanças de nomes de chaves estrangeiras em entidades para facilitar na hora de transformar em SQL.

• Projeto Físico

SQL

```
-- 1. Tabela Principal (Superclasse)
CREATE TABLE usuario (
    id_usuario SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    telefone VARCHAR(20),
    data_nasc DATE,
    sexo VARCHAR(10) -- Ou CHAR(1)
);

-- 2. Especializações (Subclasses)
```

```

-- PK é também uma FK para a tabela usuario
CREATE TABLE usuario_premium (
    id_usuario INT PRIMARY KEY,
    forma_assinatura VARCHAR(50),
    data_assinatura DATE DEFAULT CURRENT_DATE,
    CONSTRAINT fk_premium_usuario FOREIGN KEY (id_usuario) REFERENCES
usuario(id_usuario) ON DELETE CASCADE
);

CREATE TABLE administrador (
    id_usuario INT PRIMARY KEY,
    id_admin VARCHAR(20) UNIQUE NOT NULL,
    CONSTRAINT fk_admin_usuario FOREIGN KEY (id_usuario) REFERENCES
usuario(id_usuario) ON DELETE CASCADE
);

CREATE TABLE usuario_basico (
    id_usuario INT PRIMARY KEY,
    CONSTRAINT fk_basico_usuario FOREIGN KEY (id_usuario) REFERENCES
usuario(id_usuario) ON DELETE CASCADE
);

-- 3. Entidade Livro
CREATE TABLE livro (
    id_livro SERIAL PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    autor VARCHAR(100),
    genero VARCHAR(50),
    colecao VARCHAR(100),
    popularidade INT DEFAULT 0,
    data_publicacao DATE,
    media_avaliacao DECIMAL(3,2) DEFAULT 0.0 -- Atributo derivado
);

```

-- 4. Relacionamentos e Entidades Associativas

```
CREATE TABLE historico_leitura (  
    id_historico SERIAL PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    id_livro INT NOT NULL,  
    status_leitura VARCHAR(50),  
    data_leitura TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_hist_usuario FOREIGN KEY (id_usuario) REFERENCES  
usuario(id_usuario),  
    CONSTRAINT fk_hist_livro FOREIGN KEY (id_livro) REFERENCES  
livro(id_livro)  
);
```

```
CREATE TABLE avaliacoes (  
    id_avaliacoes SERIAL PRIMARY KEY,  
    id_livro INT NOT NULL,  
    id_usuario INT NOT NULL, -- Quem avaliou  
    nota INT CHECK (nota >= 1 AND nota <= 5),  
    texto TEXT,  
    data_avaliacao DATE DEFAULT CURRENT_DATE,  
    CONSTRAINT fk_ava_livro FOREIGN KEY (id_livro) REFERENCES  
livro(id_livro),  
    CONSTRAINT fk_ava_usuario FOREIGN KEY (id_usuario) REFERENCES  
usuario(id_usuario)  
);
```

-- 5. Financeiro e Dados Bancários

```
CREATE TABLE dados_bancarios (  
    id_dados_bancarios SERIAL PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    banco VARCHAR(50),  
    num_conta VARCHAR(20),  
    tipo_conta VARCHAR(20),  
    agencia VARCHAR(10),
```

```

        CONSTRAINT fk_dados_usuario FOREIGN KEY (id_usuario) REFERENCES
usuario(id_usuario)
);

```

```

CREATE TABLE pagamento (
    id_pagamento SERIAL PRIMARY KEY,
    id_dados_bancarios INT NOT NULL,
    meios_pagamento VARCHAR(50),
    data_inicio DATE,
    data_venc DATE,
    valor_total DECIMAL(10,2), -- Atributo derivado (soma das parcelas)
    CONSTRAINT fk_pag_dados FOREIGN KEY (id_dados_bancarios) REFERENCES
dados_bancarios(id_dados_bancarios)
);

```

-- 6. Entidade Fraca (Parcela)

-- Chave primária composta (id_pagamento + num_parcela)

```

CREATE TABLE parcela (
    id_pagamento INT,
    num_parcela INT,
    valor_parcela DECIMAL(10,2),
    juros DECIMAL(5,2) DEFAULT 0,
    data_venc DATE,
    data_pag DATE,
    hora_pag TIME,
    PRIMARY KEY (id_pagamento, num_parcela),
    CONSTRAINT fk_parcela_pagamento FOREIGN KEY (id_pagamento) REFERENCES
pagamento(id_pagamento) ON DELETE CASCADE
);

```

-- 7. Tabela de Gerência (N:N entre Admin e Livro)

```

CREATE TABLE gerencia (
    id_usuario_admin INT,
    id_livro INT,

```

```

PRIMARY KEY (id_usuario_admin, id_livro),
CONSTRAINT fk_gerencia_admin FOREIGN KEY (id_usuario_admin) REFERENCES
administrador(id_usuario),
CONSTRAINT fk_gerencia_livro FOREIGN KEY (id_livro) REFERENCES
livro(id_livro)
);

```

• Dependências Funcionais e Normalização

Tabela **livro**: Problema de **1FN**:

Os atributos autor e gênero são multivalorados (um livro pode ter vários autores e vários gêneros). No modelo relacional, não pode ter listas dentro de uma coluna.

É obrigatório criar tabelas para Autor e Gênero, e tabelas associativas (N:M) ligando-as ao livro.

2FN: nossa entidades fortes não contém múltiplas chaves, portanto passa no 2FN.

3FN: Nenhuma tabela possui atributos transitivos, que não esteja fortemente ligado com a chave, logo, está no 3FN.

Após normalização:

livro = {pk: id_livro, titulo, colecao, data_publicacao}

autor = {pk: id_autor, nome}

genero = {pk: id_genero, nome_genero}

livro_autor = {pk/fk: id_livro, pk/fk: id_autor} (Tabela associativa)

livro_genero = {pk/fk: id_livro, pk/fk: id_genero} (Tabela associativa)

SQL após a normalização:

None

```

-- Definição das Entidades Principais
-- 1. Tabela Autor

```



```

CREATE TABLE autor (
    id_autor SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

-- 2. Tabela Gênero
CREATE TABLE genero (
    id_genero SERIAL PRIMARY KEY,
    nome_genero VARCHAR(50) NOT NULL
);

-- 3. Tabela Livro (Estrutura normalizada)
CREATE TABLE livro (
    id_livro SERIAL PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    colecao VARCHAR(100),
    data_publicacao DATE
);

-- Definição das Tabelas de Relacionamento (Associações N:N)

-- 4. Tabela Livro_Autor (Relacionamento entre Livro e Autor)
CREATE TABLE livro_autor (
    id_livro INT NOT NULL,
    id_autor INT NOT NULL,
    PRIMARY KEY (id_livro, id_autor),
    CONSTRAINT fk_livro_aut FOREIGN KEY (id_livro) REFERENCES livro(id_livro) ON
DELETE CASCADE,
    CONSTRAINT fk_autor_liv FOREIGN KEY (id_autor) REFERENCES autor(id_autor) ON
DELETE CASCADE
);

-- 5. Tabela Livro_Genero (Relacionamento entre Livro e Gênero)
CREATE TABLE livro_genero (
    id_livro INT NOT NULL,
    id_genero INT NOT NULL,
    PRIMARY KEY (id_livro, id_genero),
    CONSTRAINT fk_livro_gen FOREIGN KEY (id_livro) REFERENCES livro(id_livro) ON
DELETE CASCADE,
    CONSTRAINT fk_genero_liv FOREIGN KEY (id_genero) REFERENCES genero(id_genero)
ON DELETE CASCADE
);

```

• Consultas SQL

- Busca por um livro específico;

SQL

```
SELECT * FROM livro WHERE titulo = 'O Hobbit';
```

- Adição de um novo livro no sistema;

SQL

```
INSERT INTO livro (titulo, colecao, data_publicacao)
VALUES ('ds', 'Coleção Tolkien', '21/09/1937');
```

- adição de gêneros e ligação de gênero com livro, para depois fazer a buscar por gênero;

SQL

```
INSERT INTO genero(nome_genero)
VALUES ('fantasia');

INSERT INTO Genero(nome_genero)
VALUES ('Literatura Infantojuvenil');

INSERT INTO livro_genero (id_livro, id_genero)
SELECT l.id_livro, g.id_genero
FROM livro l, genero g
WHERE l.titulo = 'O Hobbit'
      AND g.nome_genero IN ('fantasia', 'Literatura Infantojuvenil');

SELECT l.titulo, l.data_publicacao
FROM livro l
JOIN livro_genero lg ON l.id_livro = lg.id_livro
JOIN genero g ON g.id_genero = lg.id_genero
WHERE g.nome_genero = 'fantasia';
```

- Criar usuário no sistema;

SQL

```
WITH novo_usuario AS (  
    INSERT INTO usuario (nome, email, telefone, data_nasc, sexo)  
    VALUES ('Carlos Silva', 'carlos@email.com', '11988888888',  
    '1995-05-15', 'Masculino')  
    RETURNING id_usuario  
)  
INSERT INTO usuario_comum (id_usuario)  
  
-- Aqui, o Carlos é um USER do banco de dados  
CREATE USER carlos_silva WITH PASSWORD 'carlos123';  
  
-- Carlos começa como um usuário comum  
GRANT grupo_comum TO carlos_silva;
```

- Adicionar avaliação de livro;

SQL

```
INSERT INTO avaliacoes (id_livro, id_usuario, nota, texto)  
VALUES (1, 10, 5, 'Excelente leitura! A construção dos personagens é  
fascinante.');
```

• Gerenciamento de Usuários

Vimos que não tinha forma de diferenciar um livro que o usuário comum pode ver dos livros que ele não pode, então decidimos adicionar o atributo nível acesso a tabela livro com valor padrão 'comum'.

E como nosso projeto deve permitir que o usuário comum veja somente livros com atributo nível_acesso = comum, percebemos que com GRANT conseguimos somente controlar a coluna, pesquisamos na internet e vimos que existe uma função RLS que consegue filtrar as linhas, também vimos que podíamos somente ter feito uma visão específica, que como foi ensinado em aula, é usado na controle de linha também, mas como achei interessante essa função, deixei as duas opções, no tópico visão eu vou colocar a outra alternativa.

SQL

```
ALTER TABLE livro ADD COLUMN nivel_acesso VARCHAR(10) DEFAULT 'comum';
-- Livros 'comum' todos veem. Livros 'premium' só o premium e o admin veem.
-- 1. Criação dos Papéis (Roles)
CREATE ROLE grupo_admins;
CREATE ROLE grupo_comum;
CREATE ROLE grupo_premium;

-- 2. Permissões de Acesso (Quem pode "entrar na sala")
-- Damos acesso de leitura para os dois grupos de usuários
GRANT SELECT ON livro, autor, genero, livro_autor, livro_genero TO
grupo_comum, grupo_premium;

-- Admins podem tudo
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO grupo_admins;

-- 3. Habilitar a Segurança de Linha (RLS)
ALTER TABLE livro ENABLE ROW LEVEL SECURITY;

-- 4. Criar as Políticas (Quem pode "ver os móveis")

-- Política para o Grupo Comum: Só vê se acesso = 'comum'
CREATE POLICY politica_ver_livros_comuns
ON livro
FOR SELECT
TO grupo_comum
USING (nivel_acesso = 'comum'); -- Use o nome da coluna que você criou (ex:
nivel_acesso)

-- Política para o Grupo Premium: Vê tudo (true)
CREATE POLICY politica_ver_tudo_premium
ON livro
FOR SELECT
TO grupo_premium
```

```
USING (true);

-- Política para o Grupo Admin: Vê e faz tudo
CREATE POLICY politica_admin_total
ON livro
TO grupo_admins
USING (true)
WITH CHECK (true);
```

• Visões (Views)

1-

```
SQL
CREATE VIEW view_livro_comum AS
SELECT
    id_livro,
    titulo,
    colecao,
    data_publicacao,
    nivel_acesso
FROM livro
WHERE nivel_acesso = 'comum';
```

Justificativa

A view **view_livro_comum** foi criada para mostrar somente os livros que o usuário básico pode acessar. Dessa forma, **view_livro_comum** o sistema consegue separar os livros comuns dos livros premium de maneira mais simples e organizada. Além disso, a view evita a necessidade de fazer o filtro dos livros em todas as consultas do sistema, facilitando o uso e a manutenção do banco de dados.

2-

```
SQL
CREATE VIEW view_historico_usuario AS
SELECT
    u.nome AS usuario,
    l.titulo AS livro,
    h.status_leitura,
    h.data_leitura
FROM historico_leitura h
JOIN usuario u
    ON h.id_usuario = u.id_usuario
JOIN livro l
    ON h.id_livro = l.id_livro;
```

Justificativa

A view de histórico de leitura foi criada para facilitar a visualização das informações de leitura dos usuários. Com ela, é possível ver de forma mais organizada os usuários, os livros lidos e as datas das leituras em uma única consulta.

• Organização Física e Indexação

Criação de índices:

-- Índice para busca de livros pelo título

```
SQL
CREATE INDEX idx_titulo_livro

ON livro(titulo);
```

-- Índice para consultas no histórico de leitura por usuário

```
SQL
CREATE INDEX idx_historico_usuario

ON historico_leitura(id_usuario);
```

-- Índice para buscas de avaliações por livro

```
SQL
CREATE INDEX idx_avaliacao_livro

ON avaliacoes(id_livro);
```

-- Índice para consultas de pagamentos dos usuários

```
SQL
CREATE INDEX idx_pagamento_dados

ON pagamento(id_dados_bancarios);
```

Justificativa baseada em desempenho:

Os índices foram criados para melhorar a velocidade das consultas realizadas com mais frequências no sistemas, como busca de livro, histórico de leitura, avaliações, pagamento. Sem índices o banco precisaria verificar todos os registros das tabelas, o que os deixaria as consultas mais lentas conforme o sistema crescesse.

Justificativa de uso:

O índice **idx_titulo_livro** foi criado para facilitar e acelerar a busca de livros pelo título.

O índice **idx_historico_usuario** foi criado para melhorar consultas relacionadas ao histórico de leitura de cada usuário.

O índice **idx_avaliacao_livro** foi criado para tornar mais rápidas as consultas de avaliações feitas nos livros.

O índice **idx_pagamento_dados** foi criado para melhorar consultas relacionadas aos pagamentos e dados bancários dos usuários.